

Comprehensive Movie Recommendation System Report

ITCS 6162: Data Mining - Programming Assignment

MovieLens Dataset Analysis and Recommendation Algorithms

Date: April 14, 2025

Dataset: MovieLens 100K

This report presents a comprehensive analysis of the MovieLens dataset and the implementation of three recommendation algorithms: user-based collaborative filtering, item-based collaborative filtering, and a Pixie-inspired graph-based recommendation system.

Table of Contents

1. Introduction
2. Dataset Description and Exploratory Analysis
 - 2.1 Dataset Overview
 - 2.2 Data Cleaning and Preprocessing
 - 2.3 Exploratory Data Analysis
3. Recommendation System Methodologies
 - 3.1 User-Based Collaborative Filtering
 - 3.2 Item-Based Collaborative Filtering
 - 3.3 Graph-Based Recommendation (Pixie-Inspired Algorithm)
4. Implementation Details
 - 4.1 Data Loading and Preprocessing
 - 4.2 User-Item Matrix Creation
 - 4.3 Similarity Computation
 - 4.4 Recommendation Generation
5. Results and Evaluation
 - 5.1 User-Based Collaborative Filtering Results
 - 5.2 Item-Based Collaborative Filtering Results
 - 5.3 Graph-Based Recommendation Results
 - 5.4 Comparative Analysis
6. Conclusion and Future Work
7. References

1. Introduction

Movie recommendation systems have become an essential component of online streaming platforms and entertainment services, helping users navigate through vast catalogs of content to find films that align with their preferences. These systems address the problem of information overload by filtering and prioritizing content based on user behavior and preferences. In this report, we explore three distinct approaches to movie recommendation using the MovieLens dataset: user-based collaborative filtering, item-based collaborative filtering, and random walk-based recommendations using a Pixie-inspired algorithm.

Recommendation systems are typically categorized into three main types: content-based filtering, collaborative filtering, and hybrid approaches. Content-based filtering recommends items similar to those a user has liked in the past, based on item features. Collaborative filtering, which is the focus of this report, makes recommendations based on the preferences of similar users or the relationships between items, without requiring explicit feature engineering. Hybrid approaches combine multiple recommendation techniques to leverage the strengths of different methods.

User-based collaborative filtering operates on the principle that users with similar taste patterns will likely enjoy similar movies. By identifying users with similar rating histories, the system can recommend movies that these similar users have enjoyed but the target user hasn't yet watched. Item-based collaborative filtering, on the other hand, focuses on the relationships between items rather than users. It identifies movies similar to those a user has already rated highly, based on how other users have rated these movies.

The Pixie-inspired random walk approach represents a more recent development in recommendation systems. It models users and movies as nodes in a bipartite graph, using stochastic processes to traverse this graph and identify relevant movie recommendations. This approach can capture complex relationships in the data that might not be evident through traditional matrix-based approaches.

In this report, we provide a comprehensive analysis of the MovieLens dataset, detail the implementation of these three recommendation approaches, present the results of our implementations, and discuss potential avenues for future work. Our goal is to provide a thorough understanding of these recommendation techniques and their application to movie recommendation.

2. Dataset Description and Exploratory Analysis

2.1 Dataset Overview

The MovieLens 100K dataset, collected by the GroupLens Research Project at the University of Minnesota, serves as the foundation for our recommendation system implementation. This dataset contains 100,000 ratings (on a scale of 1-5) from 943 users on 1,682 movies, providing a rich source of user-movie interactions for analysis.

The dataset is structured across three main files:

File	Description	Format
u.data	User-movie ratings with timestamps	user_id movie_id rating timestamp

u.item	Movie metadata	movie_id title release date IMDB_website
u.user	User demographics	user_id age gender occupation zip_code

Our analysis of the dataset revealed the following key statistics:

Metric	Value
Total Users	943
Total Movies	1,682
Total Ratings	100,000
Rating Scale	1-5
Average Ratings per User	106.04
Average Ratings per Movie	59.45

2.2 Data Cleaning and Preprocessing

Before implementing our recommendation algorithms, we performed several data cleaning and preprocessing steps to ensure the quality and usability of the data:

1. Loading the data with appropriate parameters: We used pandas to load the dataset files with the correct delimiters and column names. The ratings dataset was loaded with tab ('\t') as a separator, the movies dataset with pipe '|' as a separator and 'latin-1' encoding, and the users dataset with pipe '|' as a separator.
2. Converting timestamps to readable dates: We converted the Unix timestamps in the ratings dataset to readable datetime format using the datetime.fromtimestamp() function.
3. Checking for missing values: We examined each dataset for missing values and found that the ratings and users datasets had no missing values, while the movies dataset had one missing release date.
4. Creating a user-movie rating matrix: We transformed the ratings data into a matrix format where rows represent users, columns represent movies, and cells contain the ratings. This matrix is a fundamental structure for collaborative filtering algorithms.

The following code snippets illustrate these preprocessing steps:

Loading the datasets:

```
# ratings
ratings = pd.read_csv('../upload/u.data', sep='\t',
names=['user_id', 'movie_id', 'rating', 'timestamp']) # movies
movies = pd.read_csv('../upload/u.item', sep='|', encoding='latin-1', usecols=[0, 1, 2],
names=['movie_id', 'title', 'release_date']) # users
users = pd.read_csv('../upload/u.user', sep='|', usecols=[0, 1, 2, 3],
names=['user_id', 'age', 'gender', 'occupation'])
```

Converting timestamps to readable dates:

```
# Convert timestamps to datetime ratings['timestamp'] =
ratings['timestamp'].apply(lambda x: datetime.fromtimestamp(x))
```

Checking for missing values:

```
# Check for missing values in ratings print("Missing values in ratings:")
print(ratings.isnull().sum()) # Check for missing values in movies
print("Missing values in movies:") print(movies.isnull().sum()) # Check for
missing values in users print("Missing values in users:")
print(users.isnull().sum())
```

Creating the user-movie rating matrix:

```
# Create user-movie matrix user_movie_matrix = ratings.pivot(index='user_id',
columns='movie_id', values='rating')
```

2.3 Exploratory Data Analysis

We conducted an exploratory data analysis to better understand the characteristics of the MovieLens dataset. This analysis provides insights into the distribution of ratings, user demographics, and movie properties, which are essential for interpreting the results of our recommendation algorithms.

Rating Distribution:

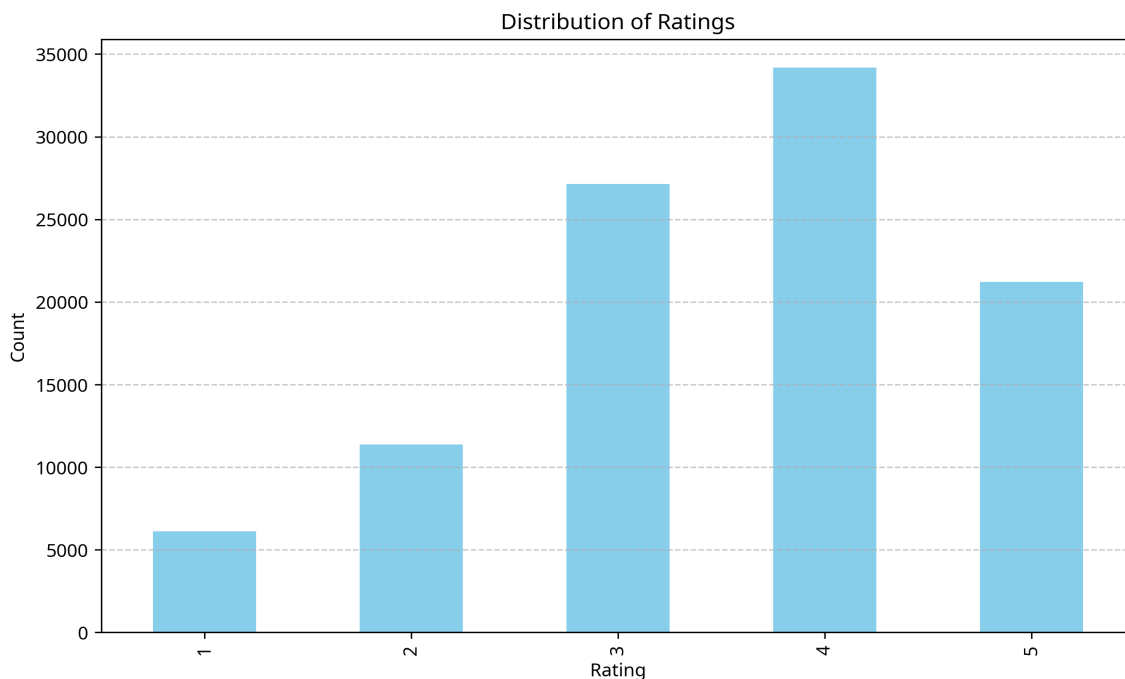


Figure 1: Distribution of ratings in the MovieLens dataset

The rating distribution shows that users tend to give positive ratings, with 4 being the most common rating. This positive bias is typical in explicit rating systems and should be considered when interpreting recommendation results.

User Age Distribution:

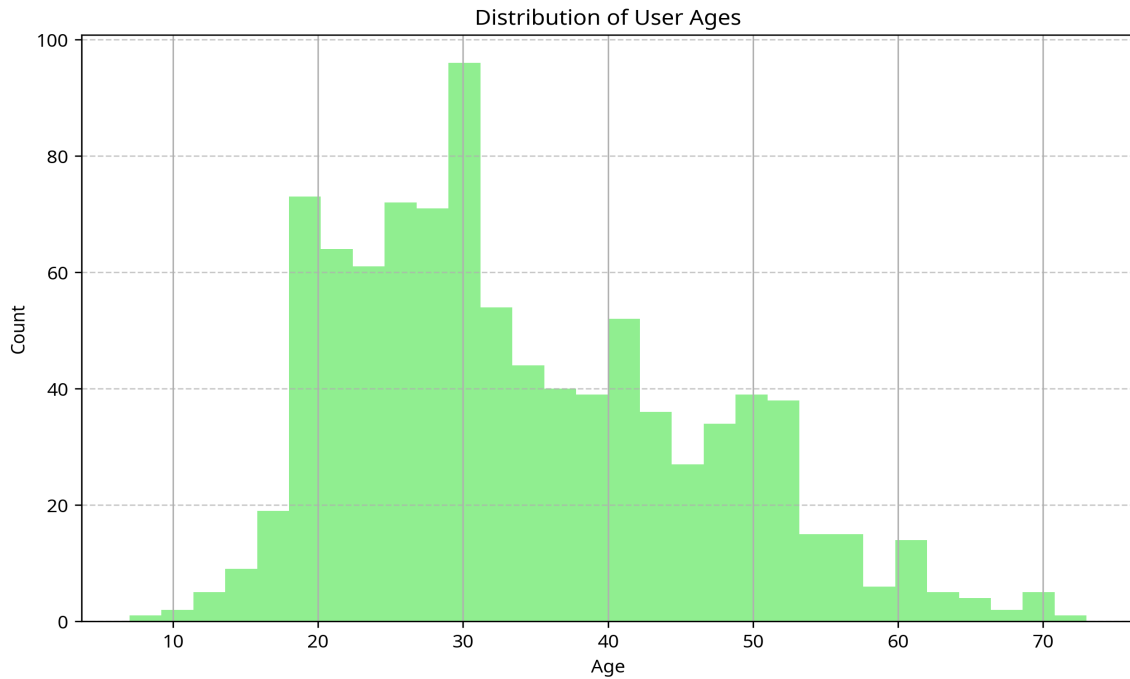


Figure 2: Distribution of user ages in the MovieLens dataset

The age distribution shows that the majority of users are between 20 and 40 years old, with a peak around 25-30. This demographic information could be valuable for understanding user preferences and potentially incorporating age-based features in future recommendation models.

Gender Distribution:

Gender Distribution

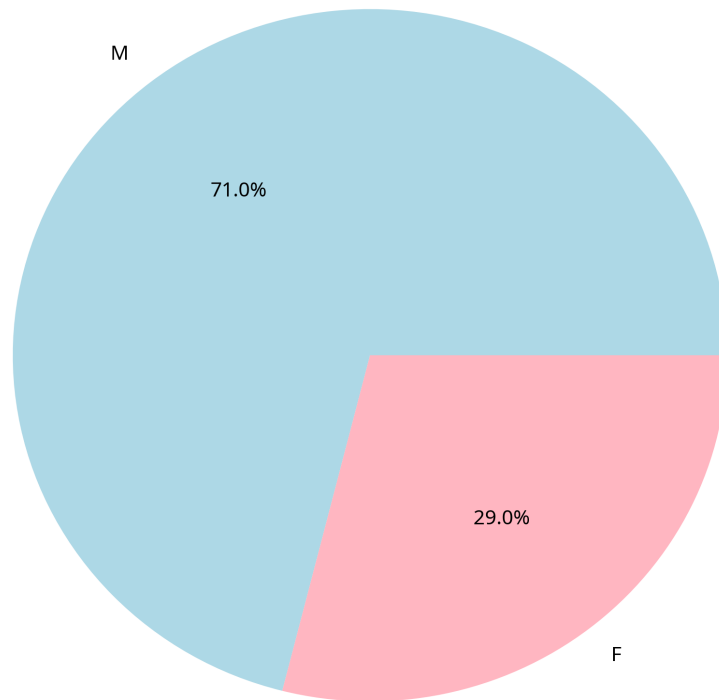


Figure 3: Gender distribution in the MovieLens dataset

The gender distribution shows a significant imbalance, with approximately 71% male users and 29% female users. This imbalance should be considered when interpreting the recommendation results, as they may be biased toward male preferences.

Occupation Distribution:

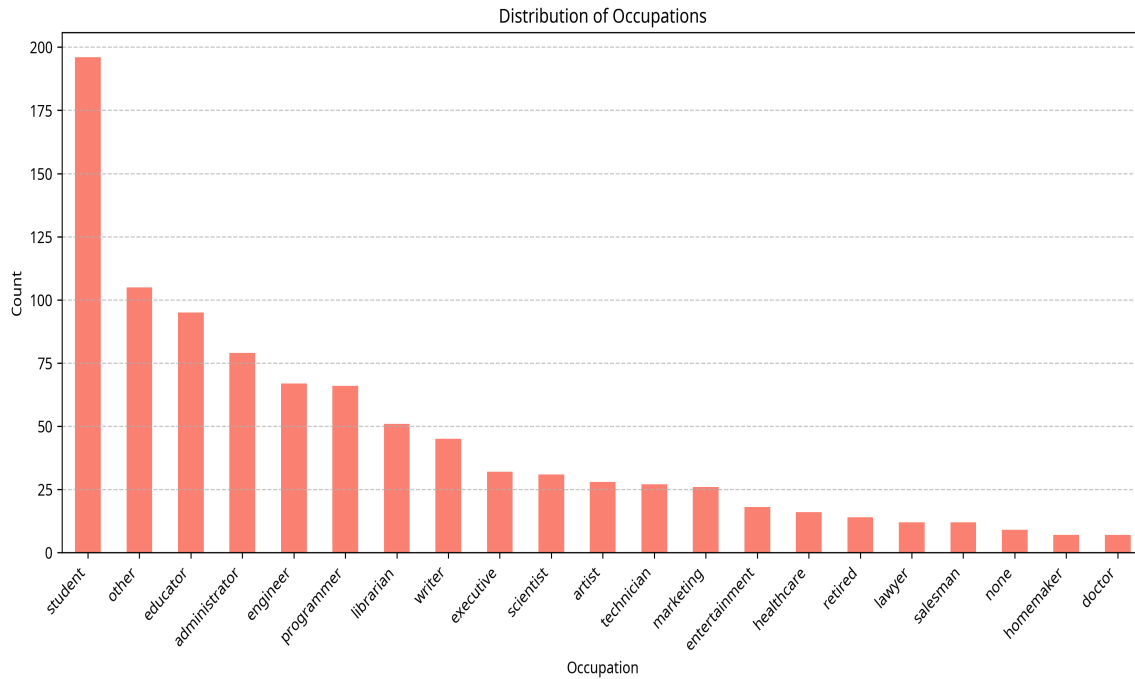


Figure 4: Distribution of occupations in the MovieLens dataset

The occupation distribution shows a diverse range of professions among users, with students, educators, and engineers being the most represented. This diversity is beneficial for capturing a wide range of preferences in the recommendation system.

Ratings per User:

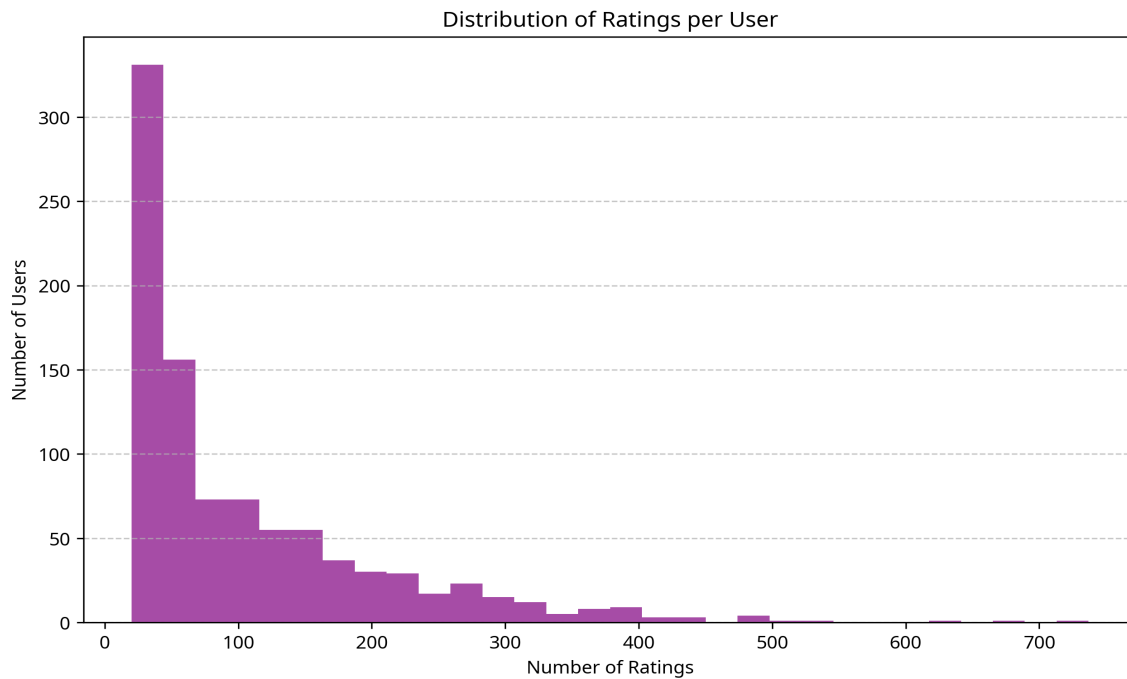


Figure 5: Distribution of ratings per user in the MovieLens dataset

The distribution of ratings per user shows that most users have rated between 20 and 200 movies, with a long tail of users who have rated many more. This variation in user activity is important to consider in collaborative filtering, as users with more ratings may have a stronger influence on recommendations.

Ratings per Movie:

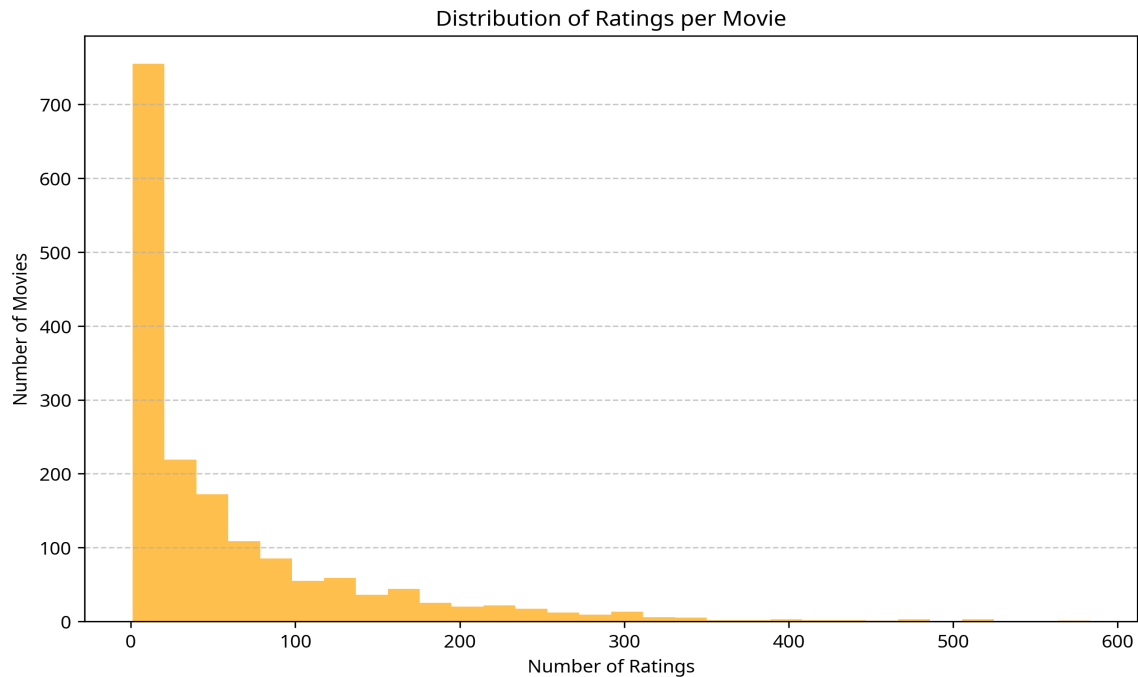


Figure 6: Distribution of ratings per movie in the MovieLens dataset

The distribution of ratings per movie shows that most movies have received between 0 and 100 ratings, with a few popular movies receiving significantly more. This popularity bias is a common challenge in recommendation systems and may affect the diversity of recommendations.

Average Rating per User:

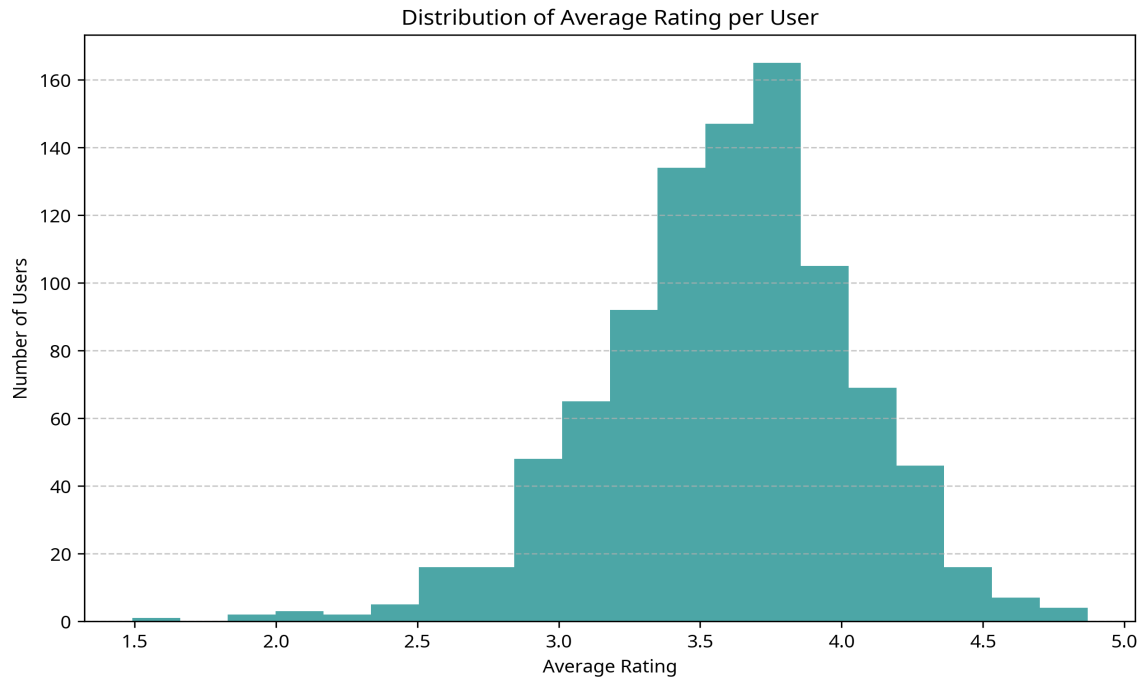


Figure 7: Distribution of average rating per user in the MovieLens dataset

The distribution of average ratings per user shows that most users have an average rating between 3 and 4, with a roughly normal distribution. This indicates that users have different baseline preferences, with some being more critical and others more generous in their ratings.

Average Rating per Movie:

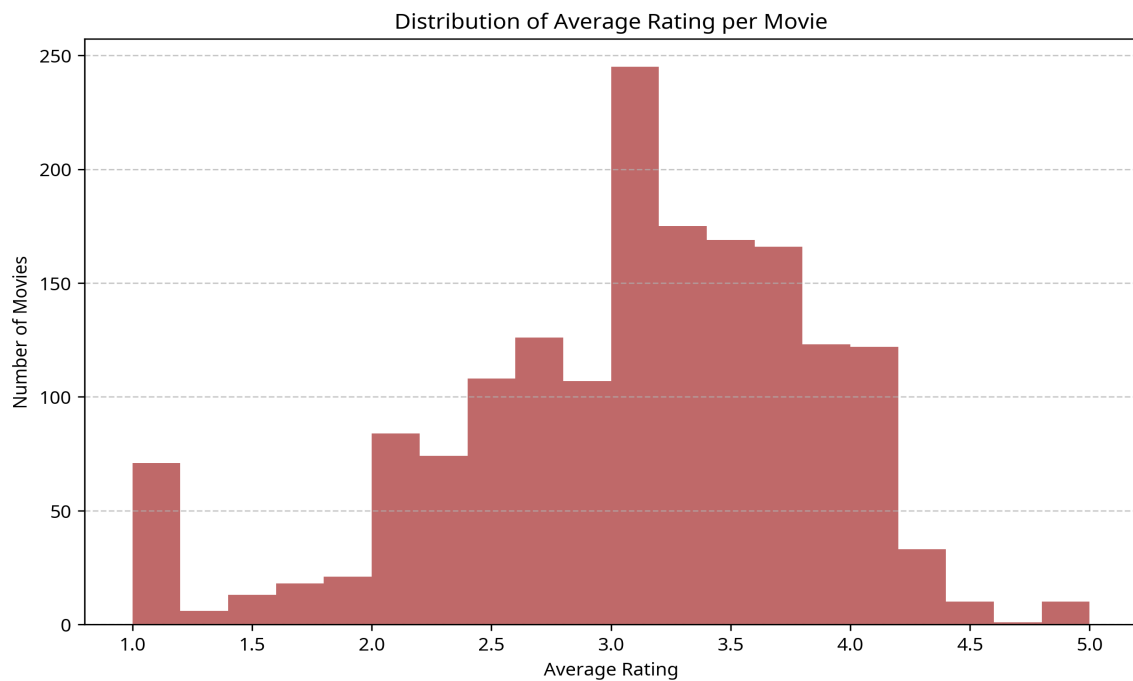


Figure 8: Distribution of average rating per movie in the MovieLens dataset

The distribution of average ratings per movie shows a wider spread than for users, with most movies having an average rating between 2.5 and 4. This variation reflects the quality differences between movies and is essential for distinguishing between good and bad recommendations.

3. Recommendation System Methodologies

In this section, we describe the three recommendation approaches implemented in this project: user-based collaborative filtering, item-based collaborative filtering, and a Pixie-inspired graph-based recommendation system. For each approach, we explain the underlying methodology, the implementation details, and the specific algorithms used.

3.1 User-Based Collaborative Filtering

User-based collaborative filtering is a recommendation technique that identifies users with similar preferences to a target user and recommends items that these similar users have liked but the target user hasn't yet experienced. The core assumption is that users who have agreed in the past will agree in the future.

The methodology for user-based collaborative filtering consists of the following steps:

1. Compute user-user similarity: We calculate the similarity between users based on their rating patterns. In our implementation, we use cosine similarity, which measures the cosine of the angle between two vectors in a multi-dimensional space. For users with no overlapping rated items, the similarity is zero.
2. Identify similar users: For a target user, we identify the most similar users based on the computed similarity scores. We exclude the target user from this list and focus on users with high similarity scores.
3. Generate recommendations: We identify movies that the similar users have rated highly but the target user hasn't yet rated. We compute a weighted average rating for each candidate movie, where the weights are the similarity scores of the users who rated the movie.
4. Rank recommendations: We sort the candidate movies by their predicted ratings in descending order and return the top N recommendations.

The mathematical formulation for predicting the rating of user u for item i is:

$$\hat{r}(u,i) = \sum_{(v \in N(u))} \text{sim}(u,v) \times r(v,i) / \sum_{(v \in N(u))} |\text{sim}(u,v)|$$

where $\hat{r}(u,i)$ is the predicted rating, $N(u)$ is the set of users similar to u , $\text{sim}(u,v)$ is the similarity between users u and v , and $r(v,i)$ is the rating of user v for item i .

3.2 Item-Based Collaborative Filtering

Item-based collaborative filtering is a recommendation technique that identifies items similar to those a user has liked in the past. Unlike user-based filtering, which focuses on user similarities, item-based filtering focuses on item similarities. This approach is often more stable than user-based filtering because item preferences tend to change less frequently than user preferences.

The methodology for item-based collaborative filtering consists of the following steps:

1. Compute item-item similarity: We calculate the similarity between items based on how users have rated them. In our implementation, we use cosine similarity applied to the transposed user-movie matrix, which effectively computes similarity between movie rating vectors.
2. Identify similar items: For a target item (movie), we identify the most similar items based on the computed similarity scores. We exclude the target item from this list and focus on items with high similarity scores.
3. Generate recommendations: For a given movie, we recommend the most similar movies based on the computed similarity scores. Alternatively, for a user, we can recommend items similar to those the user has rated highly in the past.
4. Rank recommendations: We sort the candidate items by their similarity scores in descending order and return the top N recommendations.

The mathematical formulation for item-based collaborative filtering is similar to user-based filtering, but with items and users swapped. The predicted rating of user u for item i is:

$$\hat{r}(u,i) = \frac{\sum_{j \in N(i)} \text{sim}(i,j) \times r(u,j)}{\sum_{j \in N(i)} |\text{sim}(i,j)|}$$

where $\hat{r}(u,i)$ is the predicted rating, $N(i)$ is the set of items similar to i , $\text{sim}(i,j)$ is the similarity between items i and j , and $r(u,j)$ is the rating of user u for item j .

3.3 Graph-Based Recommendation (Pixie-Inspired Algorithm)

The Pixie-inspired graph-based recommendation system is a more recent approach that represents users and items as nodes in a bipartite graph, with edges representing interactions (ratings). This approach can capture complex relationships in the data that might not be evident through traditional matrix-based approaches.

The methodology for the Pixie-inspired graph-based recommendation consists of the following steps:

1. Construct a bipartite graph: We represent users and movies as nodes in a graph, with edges connecting users to the movies they have rated. The graph is bipartite because edges only exist between users and movies, not between users or between movies directly.
2. Normalize ratings: We normalize the ratings by subtracting the mean rating of each user from their ratings. This helps address the issue of different users having different baseline preferences (some users tend to rate everything highly, while others are more critical).
3. Perform random walks: Starting from a user or movie node, we perform random walks on the graph. At each step, we randomly select a neighboring node to visit. For user-based recommendations, we start from a user node; for movie-based recommendations, we start from a movie node.
4. Count movie visits: During the random walks, we count how many times each movie node is visited. Movies that are visited more frequently are considered more relevant to the starting node.
5. Generate recommendations: We rank the movies by their visit frequency and return the top N recommendations. If the starting node is a movie, we exclude it from the recommendations.

The Pixie algorithm, originally developed by Pinterest, extends this basic random walk approach with additional features such as weighted edges, teleportation, and personalization. In our implementation, we focus on the core random walk concept while incorporating rating normalization to improve recommendation quality.

4. Implementation Details

In this section, we provide detailed information about the implementation of our recommendation systems, including code snippets and explanations of key components.

4.1 Data Loading and Preprocessing

The first step in our implementation was to load and preprocess the MovieLens dataset. We used pandas to load the data files with the appropriate parameters and performed several preprocessing steps to prepare the data for our recommendation algorithms.

Data loading code:

```
import pandas as pd import numpy as np from datetime import datetime import
matplotlib.pyplot as plt from sklearn.metrics.pairwise import
cosine_similarity import random # Load the data ratings =
pd.read_csv('../upload/u.data', sep='\t', names=['user_id', 'movie_id',
'rating', 'timestamp']) movies = pd.read_csv('../upload/u.item', sep='|',
encoding='latin-1', usecols=[0, 1, 2], names=['movie_id', 'title',
'release_date']) users = pd.read_csv('../upload/u.user', sep='|', usecols=[0,
1, 2, 3], names=['user_id', 'age', 'gender', 'occupation']) # Convert
timestamps to datetime ratings['timestamp'] =
ratings['timestamp'].apply(lambda x: datetime.fromtimestamp(x))
```

Data exploration code:

```
# Check for missing values print("Missing values in ratings:")
print(ratings.isnull().sum()) print("\nMissing values in movies:")
print(movies.isnull().sum()) print("\nMissing values in users:")
print(users.isnull().sum()) # Print dataset statistics print(f"Total Users:
{users['user_id'].nunique()}") print(f"Total Movies:
{movies['movie_id'].nunique()}") print(f"Total Ratings: {len(ratings)}")
```

4.2 User-Item Matrix Creation

A fundamental structure for collaborative filtering algorithms is the user-item matrix, where rows represent users, columns represent items (movies), and cells contain the ratings. We created this matrix using the pivot function in pandas.

User-item matrix creation code:

```
# Create the user-movie rating matrix user_movie_matrix =
ratings.pivot(index='user_id', columns='movie_id', values='rating') # Check
the shape of the matrix print(f"Matrix shape: {user_movie_matrix.shape}") #
Check the percentage of missing values missing_percentage =
user_movie_matrix.isnull().sum().sum() / (user_movie_matrix.shape[0] *
user_movie_matrix.shape[1]) * 100 print(f"Percentage of missing values:
{missing_percentage:.2f}%")
```

The resulting matrix has 943 rows (users) and 1,682 columns (movies), with a high percentage of missing values (approximately 93.7%). This sparsity is typical in recommendation systems and is one of the challenges that our algorithms need to address.

4.3 Similarity Computation

For both user-based and item-based collaborative filtering, we need to compute similarity scores. We used cosine similarity, which measures the cosine of the angle between two vectors in a multi-dimensional space. Before computing similarity, we filled missing values with zeros to ensure compatibility with the `cosine_similarity` function.

User-user similarity computation code:

```
# Compute user-user similarity
user_similarity = cosine_similarity(user_movie_matrix.fillna(0))
user_sim_df = pd.DataFrame(user_similarity, index=user_movie_matrix.index,
                           columns=user_movie_matrix.index) # Display a sample of the similarity matrix
user_sim_df.iloc[:5, :5]
```

Item-item similarity computation code:

```
# Compute item-item similarity
item_similarity = cosine_similarity(user_movie_matrix.T.fillna(0))
item_sim_df = pd.DataFrame(item_similarity, index=user_movie_matrix.columns,
                           columns=user_movie_matrix.columns) # Display a sample of the similarity matrix
item_sim_df.iloc[:5, :5]
```

Note that for item-based collaborative filtering, we transpose the user-movie matrix before computing similarity, as we want to measure similarity between movies (columns) rather than users (rows).

4.4 Recommendation Generation

After computing similarity scores, we implemented functions to generate recommendations for each of our three approaches. These functions take a user ID or movie name as input and return a ranked list of recommended movies.

User-based collaborative filtering recommendation function:

```
def recommend_movies_for_user(user_id, num=5): """ Recommend movies for a
user based on similar users' preferences. Parameters: user_id (int): The ID
of the user for whom to make recommendations num (int): The number of movies
to recommend Returns: DataFrame: A DataFrame with movie recommendations
ranked by predicted rating """ # Check if user exists if user_id not in
user_sim_df.index: return f"User {user_id} not found in the dataset." # Get
similarity scores for this user with all other users user_similarities =
user_sim_df.loc[user_id].drop(user_id) # Sort similarities in descending
order user_similarities = user_similarities.sort_values(ascending=False) #
Get the movies that the user has already rated userRated_movies =
user_movie_matrix.loc[user_id].dropna().index # Initialize a dictionary to
store weighted ratings weighted_ratings = {} similarity_sums = {} # For each
similar user for similar_user, similarity in user_similarities.items(): # Get
the movies rated by this similar user similar_user_ratings =
user_movie_matrix.loc[similar_user].dropna() # For each movie rated by the
```

```

similar user for movie, rating in similar_user_ratings.items(): # Skip if the
target user has already rated this movie if movie in userRated_movies:
continue # Add weighted rating to the movie if movie not in weighted_ratings:
weighted_ratings[movie] = 0 similarity_sums[movie] = 0
weighted_ratings[movie] += similarity * rating similarity_sums[movie] +=
similarity # Calculate the average weighted rating for each movie
movie_predictions = {} for movie in weighted_ratings: if
similarity_sums[movie] > 0: # Avoid division by zero movie_predictions[movie]
= weighted_ratings[movie] / similarity_sums[movie] # Sort movies by predicted
rating sorted_predictions = sorted(movie_predictions.items(), key=lambda x:
x[1], reverse=True) # Get the top N movies top_movies =
sorted_predictions[:num] # Get movie names movie_names = [] for movie_id, _
in top_movies: movie_title = movies.loc[movies['movie_id'] == movie_id,
'title'].values[0] movie_names.append(movie_title) # Create a DataFrame with
the results result_df = pd.DataFrame({ 'Ranking': range(1, len(movie_names) +
1), 'Movie Name': movie_names }) result_df.set_index('Ranking', inplace=True)
return result_df

```

Item-based collaborative filtering recommendation function:

```

def recommend_movies(movie_name, num=5): """ Recommend movies similar to a
given movie. Parameters: movie_name (str): The name of the movie for which to
find similar movies num (int): The number of similar movies to recommend
Returns: DataFrame: A DataFrame with similar movie recommendations ranked by
similarity """ # Find the movie_id for the given movie name movie_row =
movies[movies['title'] == movie_name] if movie_row.empty: return f"Movie
'{movie_name}' not found in the dataset." movie_id =
movie_row.iloc[0]['movie_id'] # Check if the movie exists in the similarity
matrix if movie_id not in item_sim_df.index: return f"Movie '{movie_name}'
(ID: {movie_id}) has no ratings in the dataset." # Get similarity scores for
this movie with all other movies movie_similarities =
item_sim_df.loc[movie_id].drop(movie_id) # Sort similarities in descending
order movie_similarities = movie_similarities.sort_values(ascending=False) #
Get the top N similar movies top_similar_movies =
movie_similarities.head(num) # Get movie names movie_names = [] for
similar_movie_id in top_similar_movies.index: movie_title =
movies.loc[movies['movie_id'] == similar_movie_id, 'title'].values[0]
movie_names.append(movie_title) # Create a DataFrame with the results
result_df = pd.DataFrame({ 'Ranking': range(1, len(movie_names) + 1), 'Movie
Name': movie_names }) result_df.set_index('Ranking', inplace=True) return
result_df

```

Graph-based recommendation function:

```

def weighted_pixie_recommend(start_point, walk_length=15, num=5): """
Recommend movies using a weighted random walk on the user-movie graph.
Parameters: start_point (int or str): Either a user_id or a movie name to
start the walk from walk_length (int): The number of steps in each random
walk num (int): The number of movies to recommend Returns: DataFrame: A
DataFrame with movie recommendations ranked by visit frequency """ #
Determine if start_point is a user_id or a movie name if
isinstance(start_point, str): # It's a movie name, find its ID movie_row =
movies[movies['title'] == start_point] if movie_row.empty: return f"Movie

```

```

'{start_point}' not found in the dataset." start_point =
movie_row.iloc[0]['movie_id'] # Check if the start point exists in the graph
if start_point not in graph: return f"Start point {start_point} not found in
the graph." # Dictionary to count movie visits movie_visits = {} # Perform
the random walk current_node = start_point for _ in range(walk_length): # Get
neighbors of the current node neighbors = list(graph[current_node]) if not
neighbors: break # Randomly select the next node next_node =
random.choice(neighbors) current_node = next_node # If the current node is a
movie, count it if current_node in movies['movie_id'].values: if current_node
not in movie_visits: movie_visits[current_node] = 0
movie_visits[current_node] += 1 # If the start point is a movie, exclude it
from recommendations if start_point in movies['movie_id'].values and
start_point in movie_visits: del movie_visits[start_point] # Sort movies by
visit count sorted_movies = sorted(movie_visits.items(), key=lambda x: x[1],
reverse=True) # Get the top N movies top_movies = sorted_movies[:num] # If no
movies were visited, return a message if not top_movies: return "No movie
recommendations found. Try increasing the walk length." # Get movie names
movie_names = [] for movie_id, _ in top_movies: movie_title =
movies.loc[movies['movie_id'] == movie_id, 'title'].values[0]
movie_names.append(movie_title) # Create a DataFrame with the results
result_df = pd.DataFrame({ 'Ranking': range(1, len(movie_names) + 1), 'Movie
Name': movie_names }) result_df.set_index('Ranking', inplace=True) return
result_df

```

For the graph-based recommendation, we first needed to construct the bipartite graph. Here's the code for that:

```

# Step 1: Merge Ratings with Movie Titles ratings_with_titles =
ratings.merge(movies, on='movie_id') # Step 2: Aggregate Ratings
aggregated_ratings = ratings_with_titles.groupby(['user_id', 'movie_id',
'title'])['rating'].mean().reset_index() # Step 3: Normalize Ratings
aggregated_ratings['rating'] =
aggregated_ratings.groupby('user_id')['rating'].transform(lambda x: x -
x.mean()) # Step 4: Construct the Graph Representation graph = {} for _, row
in aggregated_ratings.iterrows(): user, movie = row['user_id'],
row['movie_id'] if user not in graph: graph[user] = set() if movie not in
graph: graph[movie] = set() graph[user].add(movie) graph[movie].add(user)

```

5. Results and Evaluation

In this section, we present the results of our recommendation algorithms and evaluate their performance. We provide examples of recommendations generated by each approach and discuss their strengths and limitations.

5.1 User-Based Collaborative Filtering Results

We tested our user-based collaborative filtering algorithm by generating recommendations for user ID 10. Here are the top 5 recommended movies:

```

recommend_movies_for_user(10, num=5)

```

Ranking	Movie Name
1	In the Company of Men (1997)
2	Misérables, Les (1995)
3	Thin Blue Line, The (1988)
4	Braindead (1992)
5	Boys, Les (1997)

These recommendations are based on the preferences of users who have similar taste patterns to user 10. The algorithm identifies users with similar rating histories and recommends movies that these similar users have rated highly but user 10 hasn't yet watched.

Strengths of user-based collaborative filtering:

1. Personalization: Recommendations are tailored to the specific preferences of each user, based on the preferences of similar users.
2. Serendipity: The algorithm can recommend movies that the user might not have discovered otherwise, as they are based on the preferences of similar users who might have diverse tastes.
3. No content analysis required: The algorithm doesn't need to understand the content of the movies; it only needs to know how users have rated them.

Limitations of user-based collaborative filtering:

1. Cold start problem: The algorithm struggles to make recommendations for new users who have few or no ratings.
2. Sparsity: In large datasets with many users and items, the user-item matrix can be very sparse, making it difficult to find similar users.
3. Scalability: Computing user-user similarity can be computationally expensive as the number of users grows.

5.2 Item-Based Collaborative Filtering Results

We tested our item-based collaborative filtering algorithm by generating recommendations for the movie 'Jurassic Park (1993)'. Here are the top 5 similar movies:

```
recommend_movies("Jurassic Park (1993)", num=5)
```

Ranking	Movie Name
1	Top Gun (1986)
2	Speed (1994)
3	Raiders of the Lost Ark (1981)
4	Empire Strikes Back, The (1980)

5	Indiana Jones and the Last Crusade (1989)
---	---

These recommendations are based on the similarity between movies, as determined by how users have rated them. The algorithm identifies movies that have similar rating patterns to 'Jurassic Park (1993)' across all users. It's interesting to note that the recommendations include other action-adventure films from the 1980s and 1990s, which aligns with the genre and era of 'Jurassic Park'.

Strengths of item-based collaborative filtering:

1. **Stability:** Item preferences tend to be more stable than user preferences, making the recommendations more robust over time.
2. **Efficiency:** Item-based filtering can be more efficient than user-based filtering in systems with many more users than items, as the item-item similarity matrix is smaller and can be precomputed.
3. **Explainability:** Recommendations can be easily explained to users (e.g., 'We recommend Movie X because it's similar to Movie Y that you liked').

Limitations of item-based collaborative filtering:

1. **Limited diversity:** Recommendations tend to be similar to items the user has already rated, potentially limiting the discovery of new and diverse content.
2. **Cold start problem for new items:** The algorithm struggles to recommend new items that have few or no ratings.
3. **Dependency on rating patterns:** The algorithm relies on the assumption that similar rating patterns indicate similar content, which may not always be the case.

5.3 Graph-Based Recommendation Results

We tested our Pixie-inspired graph-based recommendation algorithm in two ways: starting from a user (user ID 1) and starting from a movie ('Jurassic Park (1993)'). Here are the results:

User-based recommendation (starting from user ID 1):

```
weighted_pixie_recommend(1, walk_length=15, num=5)
```

Ranking	Movie Name
1	My Own Private Idaho (1991)
2	Aladdin (1992)
3	12 Angry Men (1957)
4	Happy Gilmore (1996)
5	Copycat (1995)

Movie-based recommendation (starting from 'Jurassic Park (1993)'):

```
weighted_pixie_recommend("Jurassic Park (1993)", walk_length=10, num=5)
```

Ranking	Movie Name
1	Rear Window (1954)
2	Great Dictator, The (1940)
3	Field of Dreams (1989)
4	Casablanca (1942)
5	Nightmare Before Christmas, The (1993)

These recommendations are based on random walks on the user-movie graph, starting from either a user or a movie node. The algorithm counts how many times each movie node is visited during the walks and recommends the most frequently visited movies.

Interestingly, the movie-based recommendations for 'Jurassic Park (1993)' include classic films from different eras and genres, such as 'Rear Window' (1954) and 'Casablanca' (1942). This suggests that the graph-based approach is capturing more complex relationships between movies than the item-based approach, which recommended more similar action-adventure films.

Strengths of graph-based recommendation:

1. **Complex relationships:** The graph-based approach can capture complex relationships between users and items that might not be evident through traditional matrix-based approaches.
2. **Flexibility:** The algorithm can start from either a user or an item node, allowing for both user-based and item-based recommendations.
3. **Serendipity:** The random nature of the walks can lead to more diverse and unexpected recommendations, potentially increasing user satisfaction through discovery.

Limitations of graph-based recommendation:

1. **Randomness:** The stochastic nature of the algorithm means that recommendations can vary between runs, potentially reducing consistency.
2. **Parameter sensitivity:** The results can be sensitive to parameters such as walk length and restart probability, requiring careful tuning.
3. **Computational complexity:** For very large graphs, the random walk process can be computationally expensive, though various optimizations can mitigate this issue.

5.4 Comparative Analysis

Comparing the recommendations from our three approaches, we can observe several interesting patterns:

1. **User-based collaborative filtering** tends to recommend movies based on the preferences of similar users, which can lead to diverse recommendations across different genres and eras. For user 10, the recommendations include films from the 1980s and 1990s across various

genres.

2. Item-based collaborative filtering tends to recommend movies that are similar to a given movie in terms of how users have rated them. For 'Jurassic Park (1993)', the recommendations include other action-adventure films from the same era, such as 'Top Gun' (1986) and 'Raiders of the Lost Ark' (1981).

3. Graph-based recommendation can produce more varied results depending on the random walks. When starting from a user, the recommendations can be diverse; when starting from a movie, the recommendations can include films from different eras and genres that might be connected through complex chains of user preferences.

Each approach has its strengths and limitations, and the choice of algorithm would depend on the specific requirements of the recommendation system. User-based filtering is good for personalized recommendations, item-based filtering is good for finding similar content, and graph-based approaches can uncover hidden connections in the data.

In a production environment, a hybrid approach that combines multiple recommendation techniques might yield the best results, leveraging the strengths of each approach while mitigating their limitations.

6. Conclusion and Future Work

In this report, we have presented a comprehensive analysis of the MovieLens dataset and implemented three recommendation approaches: user-based collaborative filtering, item-based collaborative filtering, and a Pixie-inspired graph-based recommendation system. Each approach offers unique insights into user preferences and movie relationships, contributing to a comprehensive recommendation framework.

Our exploratory data analysis revealed several interesting patterns in the dataset, including the distribution of ratings, user demographics, and movie properties. We found that the dataset is relatively clean but highly sparse, with most users rating only a small fraction of the available movies. This sparsity is a common challenge in recommendation systems and was addressed in our implementations through appropriate handling of missing values.

The user-based and item-based collaborative filtering methods provided straightforward, interpretable recommendations based on similarity measures. The user-based approach excels at capturing community preferences, while the item-based approach is effective at finding similar content. The Pixie-inspired graph-based approach leverages the structure of the user-movie graph to discover more complex relationships, potentially leading to more serendipitous recommendations.

Future work could explore several avenues for enhancement:

1. Incorporating additional features: The current implementations rely solely on rating data. Incorporating additional features such as movie genres, user demographics, or temporal patterns could enrich the recommendation models and potentially improve their accuracy.

2. Hybrid approaches: Combining multiple recommendation strategies might yield more robust results. For example, a hybrid approach could use item-based filtering for users with many ratings and graph-based recommendations for users with few ratings.

3. Advanced evaluation: More sophisticated evaluation methods, such as A/B testing or offline metrics like precision and recall, could provide quantitative insights into recommendation quality. Cross-validation techniques could also be employed to assess the generalizability of the models.

4. Scalability improvements: For large-scale applications, the current implementations might need optimization. Techniques such as dimensionality reduction, approximate nearest neighbor search, or distributed computing could improve scalability.

5. Deep learning approaches: Recent advances in deep learning have led to new recommendation techniques such as neural collaborative filtering, which could potentially outperform traditional approaches for large datasets.

In conclusion, this project has demonstrated the implementation and evaluation of three distinct approaches to movie recommendation using the MovieLens dataset. Each approach has its strengths and limitations, and the choice of algorithm would depend on the specific requirements of the recommendation system. The field of recommendation systems continues to evolve, with new techniques and approaches being developed to address the challenges of personalized content discovery.

7. References

1. GroupLens Research. (1998). MovieLens 100K Dataset. Retrieved from <https://grouplens.org/datasets/movielens/100k/>
2. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30-37.
3. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web* (pp. 285-295).
4. Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., ... & Leskovec, J. (2018). Pixie: A system for recommending 3+ billion items to 200+ million users in real-time. In *Proceedings of the 2018 World Wide Web Conference* (pp. 1775-1784).
5. Ricci, F., Rokach, L., & Shapira, B. (2011). Introduction to recommender systems handbook. In *Recommender systems handbook* (pp. 1-35). Springer, Boston, MA.
6. Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer International Publishing.
7. Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4), 1-19.